

CDS View 課程練習 5：CDS View 分層設計——Interface View → Composite View

Lecture

這一課要教什麼

cds01 ~ cds04 陸續建了好幾個 CDS View，但都只有單層。這一課要正式把「分層設計」這個概念定名，並且用一個實戰範例，直接解決 cds02 留下的一個真實限制——還記得嗎？cds02 發現這系統的 **CASE WHEN** 判斷條件不能引用同一句 **SELECT** 清單裡算好的別名，所以 **OccupancyStatus** 的分類邏輯被迫改成只用 **seatsocc / seatsmax** 的直接比較，沒辦法真的依「佔位率百分比」分類。cds02 講義當時預告：「把計算結果放進下一層 View，就能在那一層的 **CASE WHEN** 引用它」——這一課就是把這句話兌現。

命名慣例：I_ / C_ / P_ / R_

SAP 官方（跟這門課沿用的慣例）用物件名稱前綴表達 CDS View 在分層架構裡的角色：

前綴	全名	角色
I_	Interface View	最貼近底層資料表的一層，職責是把技術欄位包裝成有語意的欄位、宣告好可能用到的 Association，但不做特定業務場景才需要的加工（例如特定報表才要的分類邏輯）。設計成可以被多個不同用途的上層 View 重複使用。
C_	Composite View（有時也稱 Consumption View）	疊在一個或多個 Interface View 之上，開始做特定業務場景才需要的加工——引用 Interface View 公開的 Association、做進一步的分類/聚合。
P_	Projection View	RAP（RESTful Application Programming Model）情境專用，疊在 Interface View 之上、給 Behavior Definition 使用的投影層（這門課不深入，RAP 課程 rap02 有教）。
R_	Query View / Report View	給分析報表或 Fiori Elements 這類最終消費端直接使用的最上層。

這一課只會用到 I_ 跟 C_ 兩層。

為什麼要分層：重用與單一職責

直接把 cds02 那種「又要算欄位、又要分類、又要串業務邏輯」的東西全部塞進一個 View，會有兩個實際的壞處：

1. **重用性差**：如果另一支程式只需要「佔位率百分比」這個原始計算值，不需要「FULL/NEARLY_FULL/AVAILABLE」這種特定分類邏輯，硬要重用這個 View 就得連分類邏輯一起接收，沒辦法只拿需要的部分。

2. **職責不單一**：一個 View 同時混雜「怎麼從底層表算出佔位率」跟「佔位率超過多少算滿座」這兩種完全不同層次的邏輯——前者是「資料怎麼來」，後者是「業務規則怎麼定」，業務規則未來很可能會變（例如滿座門檻從 95% 改成 90%），混在一起會讓修改的影響範圍不必要地擴大。

分層之後：**Interface View** 只負責「這份資料長什麼樣子、原始計算欄位是什麼、可能關聯到哪些其他資料」；**Composite View** 負責「這個特定業務場景需要的加工」。而這一課示範的案例額外證明了分層還有第三個好處：繞過『同句 **SELECT** 清單不能引用計算欄位』這個技術限制——因為 Composite View 是站在 Interface View「之上」查詢，Interface View 算好的欄位對 Composite View 來說已經是一個貨真價實的「表欄位」，不是同句清單裡的別名，**CASE WHEN** 自然可以引用。

Layer 1 : **ZI_CDS05_FLIGHT** (Interface View)

```
@AbapCatalog.sqlViewName: 'ZICDS05FLGT'
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'CDS05: Flight Interface View (Layer 1)'
define view ZI_CDS05_FLIGHT
  with parameters
    p_carrid : s_carr_id
  as select from sflight as Flight
  association [1..1] to scarr as _Carrier
    on _Carrier.carrid = Flight.carrid
{
  key Flight.carrid,
  key Flight.connid,
  key Flight.fldate,
  Flight.price,
  Flight.currency,
  Flight.seatsmax,
  Flight.seatsocc,

  cast( Flight.seatsocc as abap.decfloat34 )
    / cast( Flight.seatsmax as abap.decfloat34 )
    * 100                                as OccupancyRatePercent,

  _Carrier
}
where Flight.carrid = $parameters.p_carrid
```

這一層做的事情：① 沿用 cds04 的 Parameters 語法，讓消費端可以指定要查哪家航空公司；② 沿用 cds02 的算術運算，算出 **OccupancyRatePercent**；③ 沿用 cds03 的 Association 手法，宣告 **_Carrier** 但不消費（只公開）。注意這一層完全沒有 **CASE WHEN**——分類邏輯留給下一層。

Layer 2 : **ZC_CDS05_FLIGHT_REPORT** (Composite View)

```
@AbapCatalog.sqlViewName: 'ZCCDS05FRPT'
@AccessControl.authorizationCheck: #NOT_REQUIRED
```

```
@EndUserText.label: 'CDS05: Flight Report Composite View (Layer 2)'
define view ZC_CDS05_FLIGHT_REPORT
  with parameters
    p_carrid : s_carr_id
  as select from ZI_CDS05_FLIGHT( p_carrid: $parameters.p_carrid )
{
  key carrid,
  key connid,
  key fldate,
  price,
  currency,
  OccupancyRatePercent,

  case
    when OccupancyRatePercent >= 95 then 'FULL'
    when OccupancyRatePercent >= 80 then 'NEARLY_FULL'
    else 'AVAILABLE'
  end as OccupancyStatus,

  _Carrier.carrname as CarrierName
}
```

三個重點：

① 疊層時傳遞參數，語法用冒號 `:`，不是等號 `=`：Composite View 自己也宣告了 `p_carrid` 參數，並在 `select from` 時把它轉傳給 Layer 1：

```
as select from ZI_CDS05_FLIGHT( p_carrid: $parameters.p_carrid )
```

⚠ 這裡容易搞混：在 CDS DDL 內部（View 疊 View）傳遞參數用冒號（`p_carrid: $parameters.p_carrid`）；但在 **Open SQL**（ABAP 程式呼叫 CDS View）傳遞參數用等號（`SELECT ... FROM zi_cds05_flight( p_carrid = 'AA' ) ...`，這一課的驗證程式就是這樣寫）。兩種語法適用的場合不同，寫錯會直接語法錯誤。

② `OccupancyRatePercent` 這次可以直接被 `CASE WHEN` 引用——因為它現在是 `ZI_CDS05_FLIGHT`（Layer 1）的一個輸出欄位，對 `ZC_CDS05_FLIGHT_REPORT`（Layer 2）來說就是一個普通的來源欄位，不是同句清單裡的別名，完全沒有 cds02 遇到的限制。

③ `_Carrier.carrname` 直接被引用，觸發 **JOIN**——這是 cds03 教過的「Association 只有被引用才轉譯成 SQL JOIN」的直接應用：Layer 1 只公開了 `_Carrier`（不消費），Layer 2 才真正引用 `_Carrier.carrname`，JOIN 是在 Layer 2 查詢時才發生。

關於「Foreign Key」的一個重要澄清

CDS 課程規劃階段的草稿曾經把 `with foreign key` 語法列進這一課的內容，但實測 / 查證後要澄清：`with foreign key` 是 **DDIC define table**（建表）語法的一部分（這門課第 8 節查證過，見

.claude/rules/sap-adt-mcp.md) · 不是 **CDS View (define view)** 的語法。CDS View 表達「這筆資料關聯到另一筆資料」的方式是這一課 (跟 cds03) 教的 **Association** · 不是 Foreign Key。

兩者的關係是：Foreign Key 是資料庫底層、資料完整性層級的約束 (且如 RAP 課程已經查證過的，這系統的 DDIC Foreign Key 只在畫面輸入層級生效，Open SQL 完全不受影響)；Association 是 CDS View 查詢層級、用來表達「這裡可以走到哪些關聯資料」的語意宣告，兩者是完全不同層次的機制，不要混為一談。

Eclipse ADT 建立 CDS View：Step by Step

1. 先建 Layer 1 **ZI_CDS05_FLIGHT**：對著 **\$TMP** 套件右鍵 → New → Other ABAP Repository Object → **Data Definition** → Name **ZI_CDS05_FLIGHT** · Package **\$TMP**
2. Templates 選 **Define View (obsolete as of AS ABAP 7.57)** · Reference Object 選 **SFLIGHT**
3. 改成上面 Layer 1 的完整內容 → Ctrl+S → Activate
4. 再建 Layer 2 **ZC_CDS05_FLIGHT_REPORT**：同樣流程，但這次**不要選 Reference Object** (因為它查的是另一個 CDS View，不是底層表)，直接手動打完整內容
5. 注意 **Layer 2** 一定要在 **Layer 1** 已經啟用成功之後才能啟用 (**select from ZI_CDS05_FLIGHT(...)** 要能解析到一個已存在的物件)
6. 對 **ZC_CDS05_FLIGHT_REPORT** 用 **Data Preview** (因為有宣告 Parameters，Data Preview 畫面會先跳出一個輸入框，要求填 **p_carrid** 的值，例如 **AA**) 確認 **OccupancyStatus / CarrierName** 都正確顯示

這一課學到的東西，接下來會怎麼用

- cds06：Access Control — 這一課的 **#NOT_REQUIRED** 還沒真正做權限控管，cds06 要換成真正的 **#CHECK**
- cds07：聚合 — 這一課的 Composite View 是「一對一疊加」，cds07 會示範「多筆彙總成一筆」的聚合疊層
- cds08 (期中整合)：正式把 cds01 ~ cds07 教的技巧疊成一個完整案例

Eclipse ADT Step by Step (重點回顧)

1. Layer 1 **ZI_CDS05_FLIGHT**：以 **SFLIGHT** 為來源，含 Parameters、算術運算、Association (不消費)
2. Layer 2 **ZC_CDS05_FLIGHT_REPORT**：以 **ZI_CDS05_FLIGHT** 為來源 (不選 Reference Object，手動輸入)，含參數轉傳 (冒號語法)、引用 Layer 1 計算欄位的 CASE WHEN、消費 Association
3. Layer 1 必須先啟用成功，Layer 2 才能啟用
4. 用 Data Preview 驗證 (會跳出 Parameters 輸入框)

學習目標

- 能講出 CDS View 命名慣例 **I_ / C_ / P_ / R_** 各自代表的角色
- 能講出分層設計的兩個核心理由 (重用性、單一職責)，並能舉出這一課案例示範的第三個好處 (繞過同層 SELECT 清單不能引用別名的限制)
- 能寫出一個兩層 CDS View (Interface View → Composite View)，Composite View 正確引用 Interface View 的計算欄位跟 Association
- 能區分「CDS DDL 內部疊層傳遞參數用冒號」跟「Open SQL 呼叫 CDS View 傳遞參數用等號」這兩種語法各自的適用場合
- 知道「Foreign Key」是 DDIC 建表語法、「Association」是 CDS View 語法，兩者是不同層次的機制，不要混淆
- 知道疊層查詢時，下層物件必須先啟用成功，上層物件才能正確解析並啟用

物件清單

物件	名稱	型別
CDS Interface View (Layer 1)	ZI_CDS05_FLIGHT	DDLS/DF
CDS Composite View (Layer 2)	ZC_CDS05_FLIGHT_REPORT	DDLS/DF
驗證程式	ZR_CDS05_DEMO	PROG/P

全部物件都在 **\$TMP** 套件，已建立並啟用（讀取 active 版本內容與寫入內容逐字相符）。

動手練習物件（你自己動手建，名稱自訂，不算在上面正式的課程物件清單裡）：

物件	建議名稱	型別	對應練習
CDS Composite View（疊在 ZI_CDS05_FLIGHT 之上）	ZC_CDS05_FLIGHT_PRICE_TIER（自訂）	DDLS/DF	動手練習

動手練習

輪到你了：疊一個全新的 Layer 2 在既有的 ZI_CDS05_FLIGHT（Layer 1，已經建好不用重建）之上，物件名稱、套件 **\$TMP**，命名自訂（例如 ZC_CDS05_FLIGHT_PRICE_TIER），要求：

1. with parameters p_carrid : s_carr_id，用冒號語法轉傳給 ZI_CDS05_FLIGHT
2. 欄位清單引用 price / currency（Layer 1 已有的原始欄位）
3. 加一個新的 CASE WHEN，依 price 分類成三個價格等級（例如 PREMIUM / STANDARD / ECONOMY，門檻自己訂）——這一題故意讓你確認「引用 Layer 1 的欄位」跟「引用 Layer 1 算好的 OccupancyRatePercent」都同樣可行
4. 也引用 _Carrier.carrname，確認 Association 消費在這一層一樣正常運作

建好、啟用成功後跟我說一聲（貼程式碼或截圖都可以），我會幫你核對語法。

如果你想額外挑戰一下：試著故意把 Layer 2 的 select from 寫成 Open SQL 的等號語法（ZI_CDS05_FLIGHT(p_carrid = \$parameters.p_carrid)），實際看一次啟用失敗的錯誤訊息，再改回正確的冒號語法——親眼看過這個錯誤，比只是讀講義印象更深。

驗證方式

ZR_CDS05_DEMO 透過 programrun 無頭驗證，依序查 Layer 1（只有原始計算值，沒有分類/航空公司名稱）跟 Layer 2（分類跟航空公司名稱都正確），並確認 Layer 2 的 OccupancyStatus 確實是依 Layer 1 的 OccupancyRatePercent 正確分類出來的：

```
=== 1. Layer 1: ZI_CDS05_FLIGHT (Interface View, p_carrid = AA) ===
AA  0017 2018/10/29          96.62337662337662337662337662337662
(no CarrierName / OccupancyStatus at this layer)
AA  0017 2018/11/30          97.14285714285714285714285714285714
(no CarrierName / OccupancyStatus at this layer)
AA  0017 2019/01/01          96.1038961038961038961038961038961
```

```
(no CarrierName / OccupancyStatus at this layer)
=== 2. Layer 2: ZC_CDS05_FLIGHT_REPORT (Composite View, p_carrid = AA) ===
AA 0017 2018/10/29          96.62337662337662337662337662337662 FULL
American Airlines
AA 0017 2018/11/30          97.14285714285714285714285714285714 FULL
American Airlines
AA 0017 2019/01/01          96.1038961038961038961038961038961 FULL
American Airlines
=== 3. Layer 2 correctly derives OccupancyStatus from Layer 1 computed field? ===
MATCH: OccupancyStatus correctly derived, CarrierName populated via association, row
count 3
```

Layer 1 完全沒有 **OccupancyStatus** / **CarrierName** (因為這一層沒有分類邏輯、也沒有消費 Association) ; Layer 2 正確依 **OccupancyRatePercent** (都超過 95%) 分類成 **FULL** , **CarrierName** 也正確透過 Association JOIN 出「American Airlines」——證實兩層疊加、參數轉傳、Association 消費全部正確運作。

動手練習的驗證方式：Eclipse 啟用成功 + 用 Data Preview 看查詢結果，貼程式碼給我核對語法。

思考題

1. 如果 **ZI_CDS05_FLIGHT** 的 **OccupancyRatePercent** 計算公式以後要改 (例如改成用不同的座位統計口徑) , 需要修改幾個物件? 如果沒有分層、直接把所有邏輯塞進一個 View , 修改的複雜度會有什麼不同?
2. 這一課的 Layer 2 只疊了一層在 Layer 1 之上, 理論上可以疊第三層 (**R_** 開頭的 Report View) 在 Layer 2 之上嗎? 如果可以, 你覺得第三層適合放什麼樣的邏輯?
3. Layer 1 的 **_Carrier** Association 沒有被消費, 但 Layer 2 卻能透過 **select from ZI_CDS05_FLIGHT(...)** 之後直接寫 **_Carrier.carrname** 引用到它——這代表 Association 除了「這一層自己引用會不會轉譯成 JOIN」之外, 還有什麼特性? (提示: Association 本身有沒有跟著「被查詢」這件事一起往上層傳遞?)
4. 如果 Layer 1 的 Parameters (**p_carrid**) 改成不轉傳給 Layer 2 (也就是 Layer 2 完全不宣告 **with parameters** , 直接 **select from ZI_CDS05_FLIGHT(p_carrid: 'AA')** 寫死) , 這樣設計有什麼優缺點?

答案

見 **zi_cds05_flight.ddls.abap**、**zc_cds05_flight_report.ddls.abap**、**zr_cds05_demo.prog.abap**。SAP 端物件：**ZI_CDS05_FLIGHT** (Layer 1)、**ZC_CDS05_FLIGHT_REPORT** (Layer 2)、**ZR_CDS05_DEMO** (驗證程式)。動手練習 (疊在 **ZI_CDS05_FLIGHT** 之上的新 Layer 2) 由你在 Eclipse 動手建立, 沒有固定答案快照——建好後跟我核對即可。